

A Multi-Agent System for Detecting AI-Induced Skill Erosion in Students

1. Abstract

Most tools built to police AI use in education answer a narrow, adversarial question: was this specific submission written by a model? That framing has produced an arms race between detectors and paraphrasing tools, and it tells a teacher nothing about whether a student can actually do the work unassisted. This proposal describes a different system - one that does not try to catch AI use at all, and instead tracks, over time, the gap between what a student can produce with tool support and what the same student can produce without it. That gap, tracked across weeks rather than a single assignment, is a much more direct and much harder-to-game signal of whether learning is actually happening. The system is built as a small set of cooperating agents, each owning one narrow responsibility - trace ingestion, divergence scoring, misconception clustering, and remediation - so that the pipeline stays debuggable as it is built and extensible afterward.

2. Problem Statement and Motivation

Concern about AI dependency among students has moved well past anecdote in the last two years. A February 2026 study of computer science students found that while AI tools made ambitious projects feel more reachable, students themselves reported real anxiety about becoming dependent on them, and were already inventing their own informal guardrails to protect their skill development [1].

That anxiety has empirical backing. A 745-respondent path-analysis study traced a direct chain running from AI dependency through cognitive offloading to measurable academic skill erosion [2]. Separate research on IT and computer science students specifically found that reliance on AI coding assistants narrows a student's attention to code logic and measurably impairs debugging and problem-solving ability over time [3]. A 2026 survey spanning more than ten thousand university students reported that heavy technical exposure to LLMs has not reliably converted into skill, confidence, or trust — and that roughly two in five AI-using students describe something close to dependency [4]. A separate regional study reaches a compatible conclusion using a different student population, which suggests the pattern is not localized to one institution or country [5].

None of this is being addressed by the tools currently deployed in classrooms. Plagiarism and AI-detection systems are built to answer a single binary question at the point of submission, and that question is both adversarial (students and detectors are locked in an escalating cat-and-mouse cycle) and beside the point (a student can write every word themselves and still have learned nothing, or use AI heavily on a draft and still understand the underlying concept). The actual question institutions need answered — is this student's capability trending up, flat, or down — requires comparing performance across time and across assistance conditions. That is a state-tracking and trend-detection problem, not a one-shot classification problem, and it is why the space remains open despite years of investment in detection tooling.

3. Why This Remains Unsolved

- Detection-first tooling (Turnitin, GPTZero, and similar) optimizes for catching a single instance of AI-generated text, not for tracking capability over weeks or months.
- Adaptive-learning platforms track correctness and completion, not the gap between assisted and unassisted performance on matched tasks — so a student who always has a tool open never gets flagged, no matter how the gap widens.
- Academic research on this problem is almost entirely survey-based (self-reported anxiety, Likert-scale dependency scores). None of the sources reviewed for this proposal describe a working longitudinal

measurement system — the gap is between what has been measured in research and what has been built as a tool.

- Misconception detection at scale needs semantic clustering, not keyword or rubric matching, which is why most existing remediation is generic ('review chapter 4') rather than targeted.

4. Proposed Solution

The system is organized as four narrowly-scoped agents behind a single orchestration layer, deliberately avoiding one large prompt trying to do everything at once. Each agent is independently testable and independently replaceable.

4.1 Trace-Collector Agent

Ingests a student's work over time — code submissions, written responses, quiz attempts — and writes it into a persistent, versioned store rather than treating each submission as a stateless event. Each record is tagged with the skill it targets and whether the attempt was made under assisted or unassisted conditions.

4.2 Divergence-Scoring Agent

Compares a student's unassisted, timed performance against assisted, take-home performance on matched-skill tasks, and produces a trend rather than a single score. A widening gap over successive checkpoints is the erosion signal; a narrowing or stable gap is evidence of genuine skill transfer.

4.3 Misconception-Clustering Agent

Embeds a student's incorrect or weak attempts and clusters them by semantic similarity rather than surface keywords, surfacing the handful of underlying concepts actually causing repeated difficulty — the same clustering approach applies whether the input is a paragraph of reasoning or a block of code.

4.4 Remediation Agent

Takes the specific misconception cluster, not a generic topic tag, and retrieves or generates a targeted intervention aimed at that exact gap. Output is split by audience: a short, actionable trend summary for the teacher, and a specific next-step exercise for the student.

5. Technical Approach and Stack

- Orchestration: agents exposed as separate tools behind a FastMCP server, so each can be called, logged, and tested independently rather than folded into one prompt.
- State and memory: a vector store (Chroma or equivalent) holding embeddings of each attempt, tagged with skill, timestamp, and assistance condition.
- Scoring: a lightweight ensemble model over structured features (time taken, hint usage, correctness, similarity to prior attempts) rather than a single LLM judgment call, so the erosion signal is auditable and not just a model's opinion.
- Retrieval: RAG over a small curated set of remediation resources, scoped tightly for an initial pilot rather than attempting broad subject coverage from day one.
- Interfaces: two thin front-ends — a teacher-facing trend dashboard and a student-facing remediation view — sharing the same backend state.

6. Key Requirements

The table below separates what the system must do from the constraints it must be built under, since a requirements list that ignores its own constraints is not one a reviewer can act on.

Category	Requirement	Notes / Constraint
Functional	Ingest a student's work history (code submissions, written responses, timed vs. take-home attempts) and persist it as versioned state	Minimum viable: CSV/JSON import of a synthetic or scraped sample dataset
	Separate unassisted (timed, closed-tool) performance from assisted (open-tool) performance on matched-skill tasks	Needs at least two comparable task sets per skill tag
	Cluster recurring misconceptions using embedding similarity rather than keyword matching	Chroma or FAISS; embeddings via any open-source sentence encoder
	Generate a trend line per student per skill, not a single pass/fail score	Store at least 3 time-ordered checkpoints to show a trend, even if synthetic
	Produce a remediation suggestion tied to the specific misconception cluster, not generic advice	RAG lookup against a small curated resource set is sufficient for a demo
Non-functional	Student-level data must be handled as sensitive; no raw submission text in logs shipped to a dashboard without redaction	A required control, not an afterthought
	Each agent callable and testable in isolation (no single monolithic prompt doing all four jobs)	Use FastMCP tool boundaries per agent, matching the pattern already used in prior work
	System must degrade gracefully with sparse history (a student with 2 data points, not 200)	Early cohorts will always be small — design for this from day one
Deliverable	Working demo covering at least one full student journey end-to-end (ingest → divergence score → cluster → remediation)	Breadth across many students is less convincing than depth on one
	A teacher-facing view, even a single static page, distinct from the student-facing remediation output	The two audiences want different information

7. Implementation Roadmap

The plan below assumes a small team, with at least one member on data and scoring and one on the two front-ends throughout, rather than a strict sequential handoff between phases.

Phase	Focus	Work
Phase 1	Foundation	Freeze the skill taxonomy and dataset shape; stand up the vector store and the trace-ingestion agent; get one record flowing end to end, even with placeholder scoring.
Phase 2	Core Intelligence	Build the divergence-scoring logic comparing assisted vs. unassisted attempts; build the misconception-clustering agent on the same

Phase	Focus	Work
		embeddings; build the remediation agent behind a single orchestration layer with clear boundaries between agents.
Phase 3	Interfaces & Hardening	Build the teacher trend view and the student remediation view; validate against seeded student journeys; harden the sparse-history and contradictory-signal cases; rehearse the demo.

8. Evaluation Criteria and Success Metrics

- Correctness of the divergence signal on a seeded dataset where the erosion/non-erosion pattern is known in advance, not just plausible-looking output.
- At least one full student journey demoed end-to-end, from raw submission to a specific remediation suggestion tied to a specific misconception.
- Graceful behavior on a sparse-history student (2–3 data points), since real deployments will not start with hundreds of records per student.
- Clear separation of teacher and student outputs, judged on whether each audience gets information it can actually act on.

9. Expected Outcomes and Impact

A working prototype would give an academic institution something none of the survey-based research reviewed above has produced: an actual instrument for measuring whether AI-assisted coursework is helping or quietly hollowing out the skills it is meant to build. Beyond an initial pilot, the same architecture generalizes past code and essays to any subject where assisted and unassisted performance can be compared — lab reports, problem sets, design coursework — without redesigning the core pipeline, only the skill taxonomy feeding it.

10. Risks and Constraints

- Data availability: real longitudinal student data will not exist at initial build time; the plan relies on a seeded, synthetic dataset constructed to have a known ground-truth trend, disclosed openly as such in any demonstration.
- Privacy: any real deployment would need institutional data-handling approval; this is called out explicitly rather than deferred to a later stage.
- Scope discipline: the four-agent design is deliberately narrower than a full production system so that a complete, working slice can be shown rather than four half-built pieces.

11. References

- [1] Kenny, A., Ly, K., Tsai, M. et al. "Managing AI Dependency: How Students Are Establishing Guardrails with AI." DORA / Google, February 2026.
- [2] Miranda, J.P.P., Cruz, M.A.D., Fernandez, A.B. et al. "Erosion of Critical Academic Skills Due to AI Dependency Among Tertiary Students: A Path Analysis." 2025.
- [3] Abarzosa, V.J., Balaba, E.I. "The Negative Effects of Over-Reliance on AI Tools in IT Student Learning." Wisdom Journal of Humanities and Social Science, Vol. 2, No. 5, May 2025.

[4] Studiosity. "The AI Literacy Trap: Latest Survey from 10,330 University Students on Losing Their Minds, AI Addiction, and a Crisis of Agency." May 2026.

[5] Venugopal, K., Vakamullu, G. "Assessment of Higher Education Students' Dependency on AI Tools in an Indian Indigenized Context." IGI Global, 2025.